SE 1105 - PROGRAMMING AND PROBLEM SOLVING I

Characters and Strings

Asst. Prof. Dr. Kazım Erdoğdu

Faculty of Engineering
Department of Software Engineering
Yaşar University

Fall, 2024

Table of Contents

1. Char
2. String

Table of Contents

1. Char

2. String

Char

- A character in programming is a unit of information corresponding to a symbol.
- Letters, numerical digits, punctuation marks, whitespace characters, special symbols, etc. are all characters for C language.
- **char** data type is used to store a single character in C.

  char *variable_name*;

- Each char data type uses 1 byte in memory.
- **%c** format specifier is used for char variables.

Char

- **For example:**

```
char c = 'A';
```

- In fact, all the characters are encoded by unique and standard numerical values in the computer.
- The common standardization has been ASCII (American Standard Code for Information Interchange).
- ASCII supports the numerical digits, punctuation marks, English alphabet letters, and basic symbols.

Char

- Due to the inclusion of other languages in computer manufacturing and the extension of different symbols worldwide, new standardizations began to be used (e.g. UTF, windows, IBM, ISO, KOI, etc.).
- All of these new variants preserve the ASCII encoding but contain more symbol encodings.
- Long story short, each character in C is encoded as an integer value.
- **%c** refers to the character while **%d** refers to its numerical value.

Char

0	► 16	32	0 48	@ 64	P 80	` 96	p 112	
☺ 1	◄ 17	! 33	1 49	A 65	Q 81	a 97	q 113	
☻ 2	↕ 18	" 34	2 50	B 66	R 82	b 98	r 114	
♥ 3	‼ 19	# 35	3 51	C 67	S 83	c 99	s 115	
♦ 4	¶ 20	$ 36	4 52	D 68	T 84	d 100	t 116	
♣ 5	§ 21	% 37	5 53	E 69	U 85	e 101	u 117	
♠ 6	▬ 22	& 38	6 54	F 70	V 86	f 102	v 118	
7	↨ 23	' 39	7 55	G 71	W 87	g 103	w 119	
8	↑ 24	(40	8 56	H 72	X 88	h 104	x 120	
	9	↓ 25	) 41	9 57	I 73	Y 89	i 105	y 121
10	→ 26	* 42	: 58	J 74	Z 90	j 106	z 122	
♂ 11	← 27	+ 43	; 59	K 75	[91	k 107	{ 123	
♀ 12	∟ 28	, 44	< 60	L 76	\ 92	l 108	| 124	
13	↔ 29	- 45	= 61	M 77	] 93	m 109	} 125	
♫ 14	▲ 30	. 46	> 62	N 78	^ 94	n 110	~ 126	
☼ 15	▼ 31	/ 47	? 63	O 79	_ 95	o 111	⌂ 127	

Figure: ASCII Table

Char

```
1  #include<stdio.h>
2
3  void main(){
4     char chr;
5     printf("Enter a char: ");
6     scanf("%c",&chr);
7     printf("Character:%c Value:%d\n",chr, chr);
8
9     for(int i = 65; i < 91; i++){
10        printf("%c : %d\n",i,i);
11    }
12 }
```

Table of Contents

1. Char
2. String

Strings

- A **string** is an array of characters.

```
char name[50];
```

- The strings end with the null character (i.e. **'\0'**).

```
char name[50] = "SE1105";
```

	[0]	[1]	[2]	[3]	[4]	[5]	[6]	...	[49]
name =	'S'	'E'	'1'	'1'	'0'	'5'	'\0'	...	

Strings

- When a string is printed on the screen, the null character is not shown.

- Initializing a string literal:

```
1 char name[50] = "SE1105";
2 char name[50] = {'S','E','1','1','0','5','\0'};
3 char name[ ] = "SE1105";
4 char name[ ] = {'S','E','1','1','0','5','\0'};
```

Strings

- Strings defined as char arrays can only be modified element by element.

```
#include<stdio.h>

void main(){
   char name[10] = {'M','a','t','t','\0'};
   name[0] = 'J';
   name[1] = 'o';
   name[2] = 'h';
   name[3] = 'n';
}
```

Strings

- We can give a value to a string by user-input via **scanf()**, as well.
- We will not examine this function in this lecture.
- Briefly, **scanf** does the following in receiving the user's string input (%s):
  - Reads the characters until it sees a whitespace character (i.e. space, tab, next line, etc.) from the buffer that keeps the user input.
  - Adds **'\0'** to the end of this character list.
  - Stores these characters in a char array in the memory.
  - Links the address of this char array with the pointer variable sent to scanf.

Strings

- We need to watchfully use **scanf** function due to the possible buffer problems.
- For possible and common problems you can visit https://www.youtube.com/watch?v=Kl23Gjp_bmI.
- There is a built-in library for string operations in C called <string.h>
- We will, however, not use it at all.
- We will define our own string functions and use them.

Exercises

1. Write a C function that returns the length of a string.
2. Write a C function that capitalizes a string.
3. Write a C function that takes a string (***str***) and two characters namely c_1 and c_2 as parameters. The function must replace every occurrence of c_1 in ***str*** with c_2.
4. Write a C function that returns whether two strings are equal.
5. Write a C function that compares two strings.

Exercises

6. Write a C function that receives two strings and check whether the second one contains the first one.
7. Write a C function that receives two strings and counts the number of occurrences of the first string in the second string.
8. Write a C function that checks whether a given string is a palindrome. Palindrome strings are strings that read the same backwards as forwards.
9. Write a C function that reverses the given string.